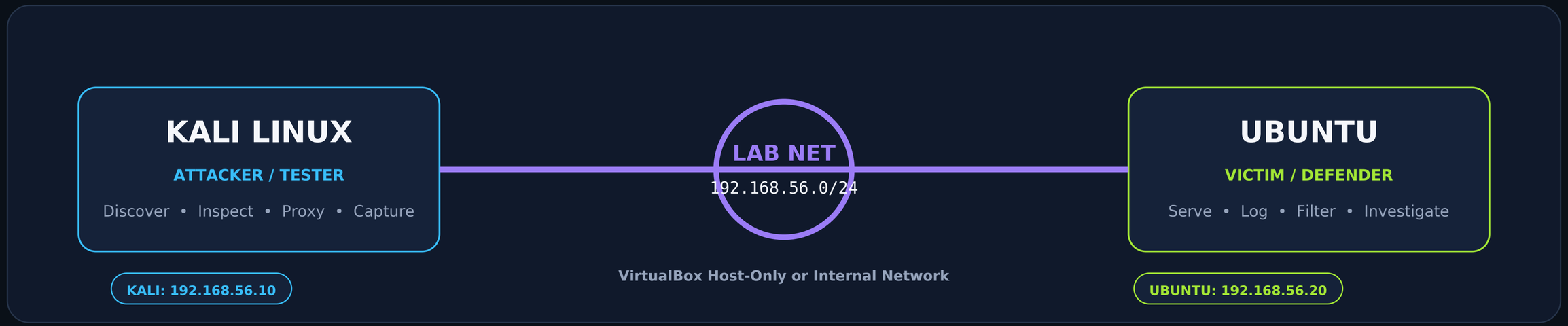
00 / LAB MAP

Kali vs Ubuntu: your practice range

One attacker VM. One victim/defender VM. One isolated network. Lots of evidence.



CORE TOOLKIT

Nmap

Find hosts, ports, and service versions.

```
sudo apt install nmap
```

Wireshark

Read packets and follow conversations.

```
sudo apt install wireshark
```

Burp Suite

Intercept and inspect HTTP requests.

```
preinstalled on Kali
```

tcpdump + curl

CLI packet capture and web testing.

```
sudo apt install tcpdump curl
```

Start with visibility, not exploitation

A repeatable workflow: identify → scan → connect → capture → verify from the defender side.

1 IDENTIFY — 2 REACH — 3 SCAN — 4 CONNECT — 5 VERIFY

KALI - DISCOVERY COMMANDS

`ip -br a`	see local interfaces
`ip route`	check route to lab
`ping -c 3 192.168.56.20`	basic reachability
`nmap -sn 192.168.56.0/24`	host discovery
`nmap -sV 192.168.56.20`	service/version scan
`nmap -p 22,80 192.168.56.20`	focused port check

UBUNTU - DEFENDER VIEW

`ip -br a`	confirm target IP
`ss -tulpn`	list listening services
`sudo journalctl -f`	watch live system logs
`sudo tcpdump -ni any host 192.168.56.10`	watch Kali traffic
`sudo ufw status verbose`	review firewall state
`last -a | head`	recent logins

PRACTICE LOOP

Run one command on Kali, then prove what happened on Ubuntu. Example: scan ports with Nmap, watch the packets in tcpdump, then compare the open ports with ss -tulpn. The goal is not only to make traffic - it is to understand the evidence it leaves behind.

SCAN → OBSERVE → EXPLAIN

See the web request from three angles

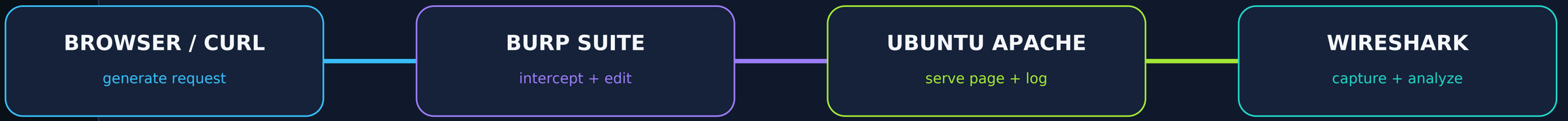
Browser view, Burp proxy view, and packet capture view should tell one consistent story.

UBUNTU - SIMPLE WEB TARGET

`sudo apt update`	refresh packages
`sudo apt install apache2`	install web server
`sudo systemctl enable --now apache2`	start + enable
`curl http://127.0.0.1`	local verification
`sudo tail -f /var/log/apache2/access.log`	watch requests

KALI - HTTP TESTING

`curl -I http://192.168.56.20`	view response headers
`curl -v http://192.168.56.20`	verbose request/response
`burpsuite`	launch interception proxy
`wireshark`	open packet analyzer
`tcpdump -ni any tcp port 80`	CLI capture of HTTP



BURP CHECKLIST

Proxy browser traffic • Intercept one request • Send it forward • Review headers and parameters • Compare with Apache access.log

WIRESHARK FILTERS

`ip.addr == 192.168.56.20` • `tcp.port == 80` • `http.request` • `tcp.flags.syn == 1`

03 / DEFEND & INVESTIGATE

Turn the victim VM into a blue-team workstation

C…uild …

UBUNTU - HARDEN & MONITOR

`sudo ufw default deny incoming`	deny unsolicited inbound
`sudo ufw allow 22/tcp`	allow SSH if needed
`sudo ufw allow 80/tcp`	allow lab web traffic
`sudo ufw enable`	turn firewall on
`sudo ufw status numbered`	review ordered rules
`sudo journalctl -u ssh -f`	watch SSH service logs

KALI - SAFE VALIDATION

`nmap -p 22,80 192.168.56.20`	confirm allowed ports
`curl http://192.168.56.20`	validate web access
`ssh user@192.168.56.20`	validate SSH login
`wireshark`	compare allowed/blocked traffic
`sudo tcpdump -ni any host 192.168.56.20`	capture evidence
`date; hostname; whoami`	document your test context

THREE MINI-LABS

01 PORT TRUTH

Start Apache. Scan from Kali. Prove the service with ss, Nmap, and access logs.

TEST • LOG • EXPLAIN

02 FIREWALL CHANGE

Block port 80 with UFW. Re-scan. Capture the difference and explain what changed.

TEST • LOG • EXPLAIN

03 HTTP EVIDENCE

Send a request through Burp. Find the same request in Wireshark and Apache access.log.

TEST • LOG • EXPLAIN

HIGH-VALUE HABIT: Before changing anything, take a VirtualBox snapshot. After every lab, save commands, screenshots, packet captures, and a one-paragraph explanation of what you learned.